

SZOFTVERTESZTELÉS

A tesztelés alapjai

FÉLÉV INFORMÁCIÓK

| Tételsor:

Előadás anyaga

| Beadandók:

1. Git repository létrehozása
2. Egységtesztek
3. Seleniumos GUI Tesztek
4. Performance teszt

| Ajánlott olvasmány:

Robert C. Martin „Uncle Bob” – Clean Code

Rex Black, Erik van Veenendaal, Dorothy Graham - Foundations of Software Testing

| ZH:

Feleletválasztós

A tárgy célja

- | Alapelvek ismerete
- | Tesztelési technikák ismerete
- | Alkalmazott tudás elsajátítása
- | Tanultak gyakorlása kódban

Miért kell tesztelnünk?

| Mars Climate Orbiter

- | Pályára állás közben megszakadt a kapcsolat
- | 125 millió dollár – 125,000,000 \$

| Ariane 5

- | 4 km magasan „eldőlt” és felrobbant
- | 375 millió dollár – 375,000,000 \$

| Crowdstrike

- | Globális szintű leállások

| A te root könyvtárad tartalma

- | Steam library frissítés közben eltűntek a beadandóid
- | Felbecsülhetetlen



Miért kell tesztelnünk?

- | Spóroljunk**
- | Bizalmat építünk a teszt tárgyában**
- | Csökkentsük a kockázatot**
- | Jobban megértsük a teszt tárgyat**
- | Validáljuk, hogy a teszt tárgya a követelményeknek megfelelően működik**
- | Teszt tárgya megfelel bizonyos jogi követelményeknek**
- | Kiváló szórakozás**

Mi a tesztelés?

A tesztelés magában foglal minden olyan tevékenységet, amelynek célja, hogy egy szoftver termékben megtaláljuk a hibákat és képesek legyünk a minőségének az értékelésére.

| Verifikáció

- |** Megfelelünk a követelményeknek
- |** Megfelelő módon készül a termék

| Validáció

- |** Megfelelünk az igényeknek
- |** Megfelelő célra készül a termék

| Nem „csak” a tesztek futtatása és kiértékelése

| Speciális gondolkodást igényel

| Emberi eredetű hiba (Error)

| Hiba (Defect) – emberi eredetű hiba eredménye, nem mindig következik belőle meghibásodás

| Meghibásodás (Failure) - megtörténik a hiba

| Kiváltó ok (Root Cause) – hibához vezető helyzet előfordulásának az oka

Minőség biztosítás (QA)

Biztosítja, hogy a termék megfeleljen a elvárásoknak. A tesztelés segít a minőség mérésében.

- | Tesztelhetőség** – milyen nehéz tesztelni a terméket
- | Karbantarthatóság** – milyen könnyen karbantartható a termék
- | Modularitás** – mennyire különülnek el a komponensek, mekkora hatása van egy komponens cseréjének
- | Megbízhatóság** – milyen sűrűn történik meghibásodás
- | Hatékonyság** – milyen jól használja ki az erőforrásokat
- | Használhatóság** – milyen könnyű használni

A TESZTELÉS ALAPELVEI

1. A tesztelés a hibák jelenlétét jelzi

- | Képes felfedni azokat
- | A szoftver minőségét és megbízhatóságát növeli

2. Nem lehet „mindent is” tesztelni

- | Általában csak a magas kockázatú és prioritású részeket teszteljük

3. Korai tesztek

- | Minél korábban jön ki a hiba, annál könnyebb és olcsóbb azt javítani
- | Már az elkészült dokumentációkat is lehet (és érdemes) tesztelni!

4. Hibák csoportosulása

- | Nincs végtelen időnk tesztelésre, emiatt a leginkább kitett és érzékeny rendszerekre kell koncentrálnunk
- | A bemenetek esetén használjunk szélső értékeket!

5. A „féregírtó” paradoxon

- | Ha ugyanazokat a tesztek futtatjuk, azok egyre kevesebb hibát fognak találni, ezért bővíteni kell őket!
- | Antibiotikumos kezelés esete – A fennmaradó 1% rezisztens, ők a legveszélyesebbek

6. A tesztelés függ a körülményektől

- | Másképp tesztelünk a környezet és idő függvényében

7. A hibátlan rendszer téveszméje

- | A hibák kijavítása nem egyenlő az igények teljesítésével!

Tesztelői tevékenységek

- | Teszttervezés**
 - | Tesztcélok**
 - | Kontextus**
 - | Stratégia**
- | Tesztfelügyelet**
 - | Tesztcélok elérése**
 - | Előrehaladás**
- | Tesztelemzés**
 - | Tesztelhető funkcionalitás**

Tesztelői tevékenységek

- | Teszt design**
 - | Tesztesetek megtervezése**
- | Teszt-implementálás**
 - | Tesztkörnyezet előkészítése**
 - | Tesztadat begyűjtése**
 - | Tesztesetek implementálása**
- | Tesztvégrehajtás**
 - | Manuális**
 - | Automata**
 - | Elvárt és tényleges eredmények összehasonlítása**
- | Teszlezárás**
 - | Archiválás**
 - | Tesztjelentés**

Résztvevők a tesztelésben

| Tesztelő

- | Manuális tesztelő és teszt automatizáló**
- | Tesztek megtervezése, megvalósítása, végrehajtása**
- | Tesztek kiértékelése**

| Tesztmenedzser

- | Teszttervezés**
- | Irányítás, kontroll**
- | Tesztlezárás**
- | Tesztjelentés**

Milyen a jó tesztelő?

- | Tisztában van a tesztelés alapvető fogalmaival**
- | Analitikus gondolkodás**
- | Odafigyel a részletekre**
- | Kíváncsi**
- | Jól tud kommunikálni**
 - | A problémára fókuszál**
 - | Kerüli a konfliktust**
- | Tisztában van a szerepekkel**

TESZTELÉS ÉS AZ SDLC

SOFTWARE DEVELOPMENT LIFE CYCLE

A fogalom

- | A szoftverrel egyidős fogalomról beszélünk
- | Életciklus
- | Igénytől az átadásig
- | Szakirodalom 7-9 "Fázisra" vagy Lépésre bontja
- | Fázisait módszertanok határozzák meg
- | „A program kódja folyamatosan változik”



2. ábra: A szoftverfejlesztés életciklusa

Kérdés: Egy szoftver életciklusa meddig tart?

MÓDSZERTANOK OSZTÁLYOZÁSA

| A vízesés modell

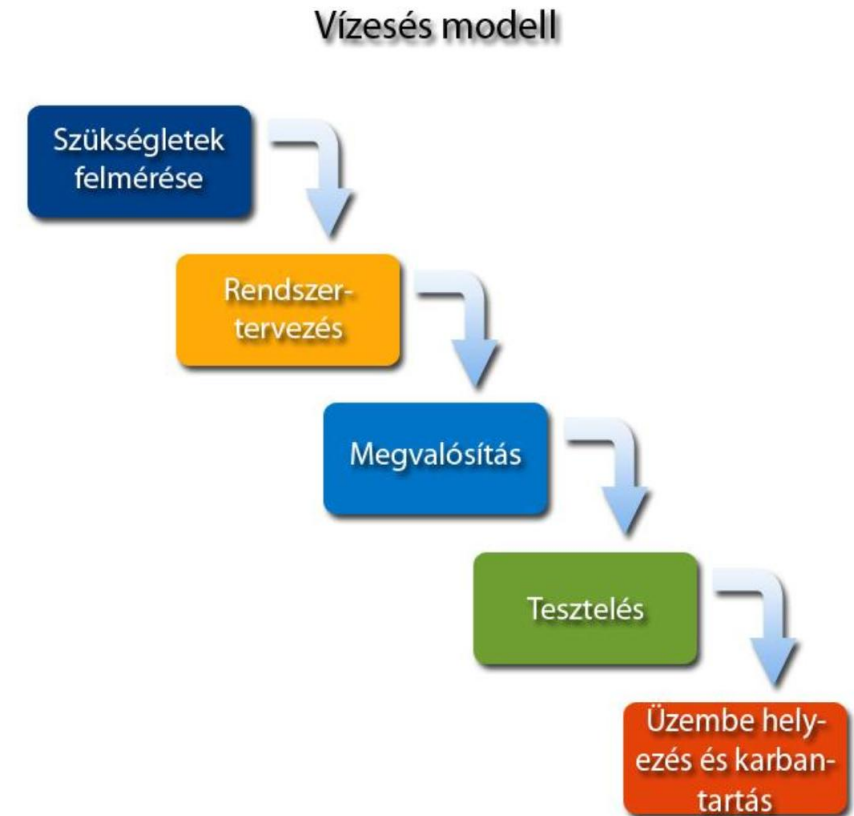
- | Lineáris
- | Strukturált
- | Jól dokumentált
- | Heavyweight
- | Nem Agilis

| Pro:

- | Erős kontroll a folyamatok felett.
- | Könnyű ütemezés

| Contra:

- | Nincs újragondolásra lehetőség
- | Szigorúan lineáris, nincs iteráció



3. ábra: A vízesés modell

MÓDSZERTANOK OSZTÁLYOZÁSA

| A V-modell

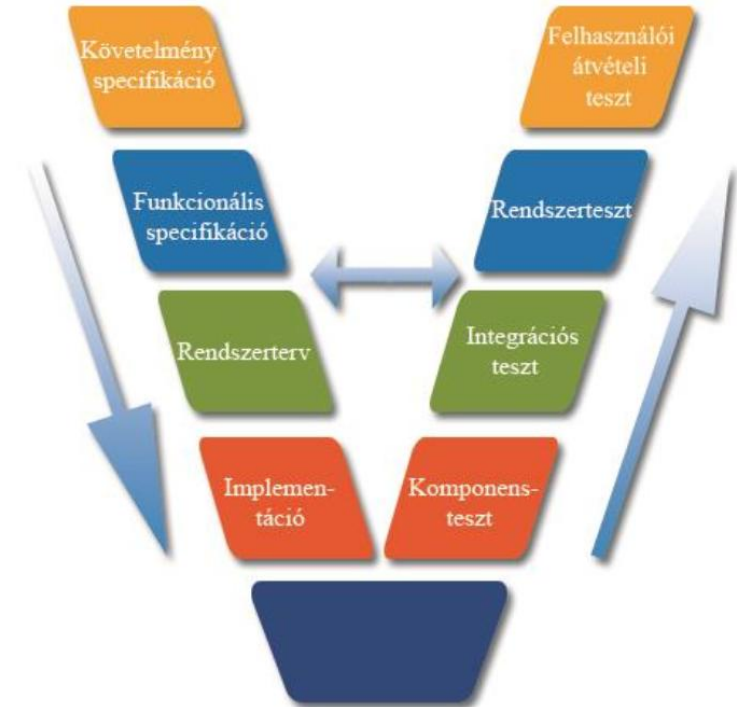
- | Lineáris
- | Strukturált
- | Jól dokumentált
- | Heavyweight
- | Tesztközpontú

| Pro:

- | Rugalmasabb, mint a vízesés modell
- | Korábban tesztlünk

| Contra:

- | Továbbra is nagyon merev
- | A követelmények változását nehezen kezeli



5. ábra: A V-modell

MÓDSZERTANOK OSZTÁLYOZÁSA

Prototípusmodell

Eldobható vagy Evolúciós prototípus

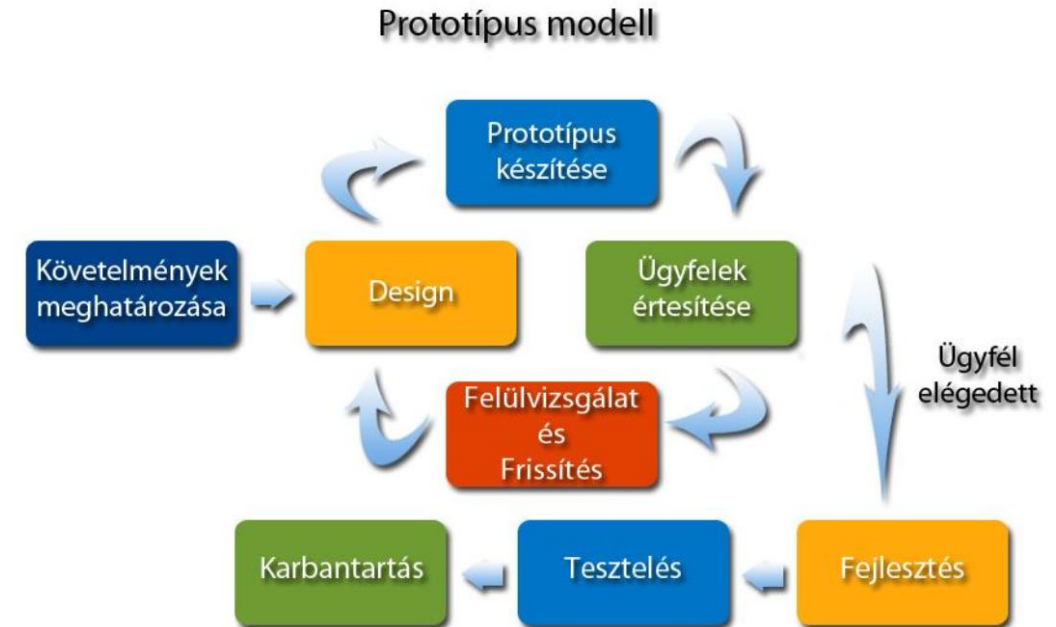
- | Általában iteratív
- | Általában nem strukturált
- | Gyakran rapid, agilis, esetleg extrém
- | Általában lightweight
- | Általában követelményközpontú

Pro:

- | Félreértések (és abból eredő pluszköltségek) minimalizálása
- | Különösen jó UI-UX kialakításnál

Contra:

- | Confirmation-bias (NEM prototípus-bias)
- | Prototípusból eredő félreértések
- | Prototípus megtartása – Bad practice



6. ábra: A prototípusmodell

MÓDSZERTANOK OSZTÁLYOZÁSA

Scrum

- | Agilis
- | Iteratív (sprint az iteráció)
- | Objektum- vagy Service orientált
- | Prototípus alapú és Rapid
- | Csapatközpontú
- | Lightweight

Pro:

- | Rendszeres kommunikáció
- | Gyorsan elsajátítható
- | Jól meghatározott szerepkörök
- | Flexibilisen illeszthető fejlesztési folyamatokhoz

Contra:

- | Nem minden Agile igazán Agile
- | Elharapódzó Backlog
- | SOK meeting



13. ábra: A SCRUM módszertan

KÖSZÖNÖM A FIGYELMET!

A tesztelés alapjai